ROBERTO FABIANO CUNHA DA SILVA

Mapeamento de Processos e Diagnóstico de Maturidade (ISO/IEC 12207 e MR-MPS-SW): Estudo de Caso do htop

Tarefa apresentada como requisito parcial para a obtenção do grau de Tecnólogo em Análise e Desenvolvimento de Sistemas.

Orientador: Prof. Me. Juan Carlos Quevedo Weimar

Rio Grande, Setembro de 2026

Milestone 1 — Mapeamento de Processos e Diagnóstico de Maturidade

Disciplina: Qualidade de Software — Tecnologia em Análise e Desenvolvimento de Sistemas **Professor:** Me. Juan Carlos Quevedo Weimar — Semestre 2026.2 **Entrega:** 21/09/2026 **Objeto de estudo:** projeto de código aberto **htop** (https://github.com/htop-dev/htop) **Normas de referência:** ISO/IEC 12207 (processos de ciclo de vida) e MR-MPS-SW (Modelo de Referência para Melhoria de Processo do Software Brasileiro)

1. Identificação do objeto de estudo

Item	**Evidência**
Produto	htop — visualizador interativo de processos para terminal
Repositório	github.com/htop-dev/htop (branch default: main)
Linguagem / licença	C99 / GPL-2.0
Histórico	Autoria original de Hisham H. Muhammad (~2004); migrado para a organização htop-dev em 17/08/2020
Versão em desenvolvimento	3.6.0-dev (configure.ac)
Última release estável	**3.5.3** (16/08/2026)
Tamanho do código	282 arquivos .c/.h, **48.726 linhas**
Comunidade	8.340 stars, 641 forks, 64 watchers, **275 contribuidores** (histórico completo)
Mantenedores	4 (Benny Baumann, Christian Göttsche, Daniel Lange, Nathan Scott) — AUTHORS
Atividade	média de **9,6 commits/semana** (janela de 50 semanas); 14 na semana da análise
Backlog	578 issues e 1.169 PRs fechados; 268 issues e 88 PRs abertos

Justificativa da escolha. O htop atende ao critério de "software real com repositório acessível para inspeção de código e análise de processo": é maduro (duas décadas), de uso massivo (presente em praticamente todas as distribuições Linux/BSD), tem governança coletiva (não é *bus factor* 1), e possui um pipeline de qualidade automatizado observável — o que permite evidenciar processos em vez de apenas supor que existem.

Método. A evidência foi coletada por inspeção direta do repositório (arquivos de governança, workflows de CI, scripts, documentação), consulta à API pública do GitHub (histórico de releases, milestones, labels, volumetria de PRs/issues) e leitura do código-fonte para confirmar práticas de verificação. Todos os números deste relatório são verificáveis nas fontes listadas na seção 7.

2. Mapeamento de Processos (ISO/IEC 12207)

A ISO/IEC 12207 organiza o ciclo de vida em quatro grupos de processos. O quadro abaixo mapeia, para cada processo relevante ao htop, a prática observada e o artefato que a comprova.

2.1 Processos de Acordo (*Agreement*)

Processo	Situação no htop	Evidência
Aquisição	**Não aplicável**	Projeto OSS sem relação formal comprador–fornecedor; dependências vêm de pacotes da distribuição.
Fornecimento (*Supply*)	**Implementado informalmente**	A entrega se dá por releases versionadas, tarball assinado por hash e pacotes nas distros.

2.2 Processos Organizacionais de Apoio ao Projeto

Processo	Situação no htop	Evidência
Gestão do modelo de ciclo de vida	**Inexistente**	Não há modelo de ciclo de vida declarado; o fluxo é contínuo/iterativo de facto.
Gestão de infraestrutura	**Implementado**	GitHub (código, issues, PRs, releases) + GitHub Actions como plataforma de CI; Dependabot semanal (.github/dependabot.yml).
Gestão de portfólio	**Não aplicável**	Produto único.
Gestão de recursos humanos	**Parcialmente implementado**	CODE_OF_CONDUCT.md; CONTRIBUTING.md como porta de entrada; labels good first issue e help wanted para onboarding; programa de mentoria implícito nas revisões.
Gestão da qualidade	**Implementado (forte)**	CI obrigatório, CodeQL, Coverity, clang-analyzer, sanitizers, style guide aplicado em lint — ver 3.3.
Gestão do conhecimento	**Implementado**	docs/styleguide.md, docs/understanding-htop-versions.md, docs/ai-contributions-policy.md, AGENTS.md, TESTPLAN, ChangeLog, man page

Processo	Situação no htop	Evidência
		(htop.1.in).

2.3 Processos Técnicos de Gestão

Processo	Situação no htop	Evidência
Planejamento de projeto	**Fraco**	Existem *milestones* por versão, mas não há plano com escopo, cronograma, estimativas ou alocação de recursos.
Avaliação e controle de projeto	**Implementado informalmente**	Milestones por release (3.5.3 fechado com 8 itens; 3.6.0 aberto com 30 abertos/33 fechados); revisão por pares em PRs.
Gestão de decisões	**Implementado informalmente**	Label needs-discussion; decisões tomadas por consenso dos mantenedores. Sem registro formal de alternativas/critérios.
Gestão de riscos	**Inexistente**	Nenhum registro de riscos, análise de probabilidade/impacto ou plano de mitigação documentado.
Gestão de configuração	**Implementado (forte)**	Git com política de versionamento documentada; tags por release; ChangeLog estruturado; releases do GitHub com tarball .tar.xz + SHA256 gerados automaticamente (build_release.yml).
Gestão da informação	**Implementado**	Issues, PRs, ChangeLog, NEWS, documentação em docs/, man page, crash reports com identificação de versão.
Medição	**Inexistente**	Não há coleta sistemática de métricas de processo ou produto (esforço, prazo, densidade de defeitos, cobertura). Os únicos indicadores são status de CI (passa/falha) e contagens do GitHub.

Processo	Situação no htop	Evidência
Garantia da qualidade	**Implementado (forte)**	O CI é o mecanismo central de GQA; ver 3.3.

2.4 Processos Técnicos

Processo	Situação no htop	Evidência
Análise de negócio/missão	**Implementado informalmente**	Missão clara e estável: monitor de processos leve, portátil e de baixo consumo (explicitada em CONTRIBUTING.md: “needs to keep a small footprint”).
Necessidades das partes interessadas	**Implementado informalmente**	Issue tracker como canal; feature request como label oficial.
Definição de requisitos	**Fraco**	Requisitos existem como issues rotuladas, sem especificação, priorização formal ou rastreabilidade bidirecional.
Definição de arquitetura	**Implementado**	Arquitetura documentada em AGENTS.md (C orientado a objetos com tabelas de métodos virtuais; abstração por plataforma em linux/, darwin/, freebsd/ etc.).
Definição de projeto (*design*)	**Implementado**	docs/styleguide.md cobre convenções de nomes, estrutura de arquivos, ordenação de includes, uso de braces, wrappers de memória.
Implementação	**Implementado**	Fluxo fork → PR → revisão → merge.
Integração do produto	**Implementado (forte)**	CI integra e compila em Ubuntu (gcc/clang, minimal/full/static), macOS, e por *cross-compilation* em **DragonFlyBSD, FreeBSD, NetBSD, OpenBSD e Solaris** — 20 jobs.
Verificação	**Implementado parcialmente**	Verificação *por ferramenta* é forte (análise estática + sanitizers); verificação *por teste* é fraca — apenas test_spec.lua (687 linhas) e o

Processo	Situação no htop	Evidência
		TESTPLAN manual.
Validação	**Parcial**	Validação indireta via feedback de usuários (issues) e roteiro manual de aceitação (TESTPLAN); não há validação formal com o cliente.
Transição	**Implementado**	Release automatizado publica tarball e hash (build_release.yml); distribuição via repositórios das distros.
Operação	**Implementado**	Ferramenta de execução local; crash handler próprio grava relatório para análise posterior (CRT.c).
Manutenção	**Implementado**	Patches de correção, correções de portabilidade e resposta a CVE (ex.: ChangeLog da 3.5.3 registra a correção do CVE-2024-37676).
Descarte	**Não aplicável**	Produto sem processo de desativação.

3. Diagnóstico de Maturidade (MR-MPS-SW)

Observação de nomenclatura: adota-se aqui a estrutura clássica do Guia Geral MR-MPS-SW (níveis G a A). Versões mais recentes do guia renomeiam **PCP** (Projeto e Construção do Produto) para **Solução Técnica (ST)** e incorporam **Gerência de Conhecimento (GKN)** ao nível E. A renomeação não altera o diagnóstico.

3.1 Avaliação por processo

Nível G — Parcialmente Gerenciado

Processo	Veredito	Justificativa baseada em evidência
GPR — Gerência de Projetos	Não aderente	Há *milestones* por versão, mas não há plano de projeto (escopo, cronograma, estimativas de esforço/prazo, alocação de recursos) nem monitoramento de desempenho contra plano.
GRE — Gerência de	Aderência fraca	Requisitos existem como

Processo	Veredito	Justificativa baseada em evidência
Requisitos		issues com labels (bug, feature request, enhancement) e são rastreáveis a PRs e milestones, mas não há *baseline* de requisitos, comprometimento formal das partes nem rastreabilidade bidirecional.

Nível F — Gerenciado

Processo	Veredito	Justificativa baseada em evidência
GCO — Gerência de Configuração	Aderente	Git + política de versionamento documentada (docs/understanding-htop-versions.md), tags por release, ChangeLog, releases com SHA256, Dependabot semanal para dependências de CI.
GQA — Garantia da Qualidade	Aderente	Pipeline obrigatório de CI (20 jobs), CodeQL em push/PR/semanal, Coverity **diário**, clang-analyzer, style guide formal, revisão por pares, CoC.
MED — Medição	Não aderente	Nenhuma coleta, armazenamento ou análise sistemática de métricas de processo ou produto.
AQU — Aquisição	Não aplicável	Sem aquisição gerenciada de produtos/serviços de terceiros.
GPP — Gerência de Portfólio	Não aplicável	Produto único.

Nível E — Parcialmente Definido

Processo	Veredito	Justificativa baseada em evidência
AMP — Avaliação e Melhoria do Processo	Aderência fraca	Há melhoria contínua *de facto* (endurecimento pós-CVE, refatorações, evolução do style guide), mas sem avaliação formal do processo nem plano de melhoria com metas mensuráveis.
DFP — Definição do Processo Organizacional	Aderência fraca	O processo existe e está parcialmente documentado (CONTRIBUTING.md, docs/styleguide.md, workflows, AGENTS.md), mas não há conjunto de

Processo	Veredito	Justificativa baseada em evidência
		processos-padrão da organização nem critérios de adaptação (*tailoring*).
GRH — Gerência de Recursos Humanos	Parcial	CoC, material de onboarding, labels de entrada e política de uso de IA (docs/ai-contributions-policy.md, v1.0 de 18/02/2026). Não há gestão formal de competências ou planos de capacitação.
GRU — Gerência de Reutilização	Aderência fraca	Há reuso incidental de código entre plataformas (generic/), sem programa de reutilização sistemático.

Nível D — Largamente Definido

Processo	Veredito	Justificativa baseada em evidência
DRE — Desenvolvimento de Requisitos	Não aderente	Sem especificação de requisitos, casos de uso ou análise formal.
PCP — Projeto e Construção do Produto	Aderência parcial	Arquitetura e convenções de projeto existem e são documentadas e aplicadas por lint; faltam projeto detalhado de unidades e critérios de aceitação por item.
ITP — Integração do Produto	Aderente	Integração automatizada contínua em 7+ plataformas e 2 compiladores, com e sem recursos opcionais (PCP, sensors, capabilities); build de release automatizado.
VER — Verificação	Aderência parcial (assimétrica)	Verificação instrumental robusta: clang-analyzer, ASan/UBSan com -fsanitize extensivo, -Wall -Wextra -Werror, _FORTIFY_SOURCE=2, valgrind com supressões próprias. Verificação por **teste funcional** é insuficiente: um único arquivo de especificação (687 linhas) para 48,7 mil linhas de código, sem medição de cobertura, **sem fuzzing** e sem testes no CI para os parsers de maior risco.
VAL — Validação	Parcial	Validação ocorre por feedback de usuários e pelo roteiro manual TESTPLAN; sem validação formal de aceitação com o cliente.

Nível C — Definido

Processo	Veredito	Justificativa baseada em evidência
ADR — Análise de Decisão e Resolução	Aderência fraca	Decisões técnicas relevantes foram tomadas (ex.: adoção de libunwind-ptrace para *backtrace*, migração para a organização htop-dev), mas sem registro formal de critérios, alternativas e justificativa.

Processo	Veredito	Justificativa baseada em evidência
GRI — Gerência de Riscos	Não aderente	Nenhum artefato de identificação, análise ou mitigação de riscos.

Níveis B e A

Processo	Veredito
GQP — Gerência Quantitativa do Projeto	Não aderente (sem base quantitativa)
GDO — Gerência de Desempenho Organizacional	Não aderente

3.2 Síntese do diagnóstico

Considerando que o MR-MPS-SW exige a implementação de **todos** os processos dos níveis inferiores para reconhecer um nível de maturidade, o resultado é:

Nível	Processos satisfeitos	Situação
G	1 de 2 (GRE fraca, GPR ausente)	**Não atende**
F	2 de 3 aplicáveis (GCO, GQA ; MED)	**Não atende**
E	0 de 4 plenamente	Não atende
D	1 de 5 plenamente (ITP)	Não atende

Nível de maturidade estimado: abaixo de G (sem conformidade plena com nenhum nível), **embora com práticas de engenharia que, isoladamente, superam as de muitos projetos de nível D/E.**

Esse aparente paradoxo é a conclusão mais relevante deste diagnóstico:

O htop possui maturidade de *produto* alta e maturidade de *processo* baixa. A qualidade técnica é sustentada por automação (CI, análise estática, sanitizers, políticas de contribuição) e não por gestão formalizada (planejamento, medição, gestão de riscos, engenharia de requisitos). O MR-MPS-SW avalia predominantemente a segunda dimensão — logo, **o modelo tende a penalizar projetos de código aberto**, que substituem a gestão por automação e revisão por pares distribuída.

A consequência prática é mensurável: as duas vulnerabilidades com CVE registradas no projeto (CVE-2008-5076, na renderização de nomes de processo; e CVE-2024-37676, no parser do htoprc) ocorreram justamente em áreas sem cobertura de teste dedicada — área que um processo de **GRI** (riscos), **MED** (medição) e **VER** (verificação por teste/fuzzing) teria priorizado.

4. Pontos fortes

1. **Verificação automatizada de primeira linha.** CodeQL em todo push/PR, Coverity diário, clang-analyzer dedicado, build com ASan/UBSan e um conjunto extenso de sanitizers, mais -Werror -Wall -Wextra e _FORTIFY_SOURCE=2.
2. **Integração multi-plataforma.** 20 jobs de CI cobrindo Linux, macOS, cinco BSD/Solaris por *cross-compilation*, dois compiladores e variações de recursos.

3. **Gerência de configuração sólida.** Versionamento documentado, releases reproduzíveis com hash, ChangeLog estruturado, milestones por versão, Dependabot.
4. **Padronização de código formal e aplicada.** docs/styleguide.md é normativo e o CI verifica parte dele (lint de whitespace, lint de config PCP).
5. **Governança madura para a categoria.** Quatro mantenedores (sem *bus factor* 1), CoC, revisão por pares obrigatória, política explícita de contribuições assistidas por IA com exigência de rastreabilidade (Assisted-by:), e assinatura DCO (Signed-off-by:).
6. **Resposta a incidentes de segurança efetiva.** Os CVEs foram corrigidos com endurecimento estrutural (validação de índices e invariantes no parser), não apenas com *patch* pontual.

5. Lacunas e recomendações (PDCA)

#	Lacuna	Impacto no modelo	Ação recomendada (Act do PDCA)
1	Ausência de fuzzing nos parsers críticos (Settings_read e leitura de /proc/<pid>/*)	VER / GRI	Adicionar *fuzzing* contínuo (ex.: libFuzzer/AFL) ao CI, priorizando entradas não confiáveis.
2	Cobertura de teste funcional muito baixa (1 spec para 48,7 mil linhas) e sem métrica de cobertura	VER / MED	Introduzir medição de cobertura e publicá-la como *gate* de PR para os módulos críticos.
3	Nenhuma métrica de processo ou produto coletada	MED	Definir conjunto mínimo (defeitos por release, tempo de resolução, cobertura, densidade de defeitos por módulo).
4	Nenhum registro de riscos	GRI	Criar registro de riscos derivado de análise de concentração (ex.: princípio de Pareto sobre módulos).
5	Requisitos sem *baseline* nem rastreabilidade bidirecional	GRE / DRE	Adotar templates de issue com critérios de aceitação e vinculo explícito issue / PR / milestone / ChangeLog.
6	Planejamento de projeto ausente (sem estimativas/cronograma)	GPR	Publicar, por milestone, escopo e critério de encerramento; medir desvio.
7	Processo organizacional não definido nem adaptável	DFP / AMP	Formalizar o processo atual (o que já existe) e instituir ciclos de avaliação com metas.

6. Aplicação do princípio de Pareto (Juran) — vínculo com o Milestone 2

Aplicado ao código do htop, o princípio 80/20 se confirma de forma ainda mais extrema que o clássico:

- Por **volume**: os 20% maiores arquivos (56 de 282) concentram 62,7% das linhas.
- Por **risco**: **11 arquivos (3,9% do total) concentram 100% dos CVEs do projeto** e todos os caminhos que combinam *parsing de entrada não confiável* com *operação privilegiada* (LinuxProcessTable.c, Settings.c, linux/Platform.c, CommandLine.c, CRT.c, Action.c, UnwindPtrace.c, TraceScreen.c, OpenFilesScreen.c, Row.c, RichString.c).

Esse recorte é a base do plano de métricas de produto e do relatório de análise estática previstos para o **Milestone 2** (ISO/IEC 25010), onde os módulos apontados serão instrumentados e analisados individualmente.

7. Referências e evidências consultadas

Repositório (inspeção direta, commit 8ff043d, 20/09/2026) - CONTRIBUTING.md, CODE_OF_CONDUCT.md, SECURITY.md, AUTHORS, ChangeLog, NEWS, TESTPLAN - docs/styleguide.md, docs/understanding-htop-versions.md, docs/ai-contributions-policy.md, AGENTS.md - .github/workflows/ci.yml, .github/workflows/build_release.yml, .github/workflows/codeql-analysis.yml, .github/workflows/coverity.yml, .github/dependabot.yml - configure.ac (versão e flags de compilação), scripts/run_valgrind.sh, test_spec.lua

API pública do GitHub (20/09/2026) - api.github.com/repos/htop-dev/htop (metadados, licença, volumetria) - .../releases, .../milestones, .../labels, .../models/contributors, .../stats/participation

Normas - ISO/IEC 12207 — *Systems and software engineering — Software life cycle processes* - MR-MPS-SW — Guia Geral MPS de Software (SOFTEX) - ISO/IEC 25010 — *Systems and software Quality Requirements and Evaluation (SQuaRE)*

Vulnerabilidades citadas - CVE-2008-5076 — escrita de nomes de processo sem sanitização de caracteres não imprimíveis - CVE-2024-37676 — acesso fora dos limites em Header_populateFromSettings alcançável via htoprc adulterado

Relatório produzido para a disciplina de Qualidade de Software (2026.2). Todos os dados quantitativos foram obtidos por inspeção direta do repositório e da API pública do GitHub na data indicada.